Politecnico di Milano

Ingegneria del Software – a.a. 2009/10

Appello del 10 Settembre 2010

Cognome
Nome
Matricola

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h10m.
6. Punteggio totale a disposizione: 100 punti nominali. La sufficienza si raggiunge indicativamente con 60 punti.

Esercizio 1: .....

Esercizio 2: .....

Esercizio 3: .....

Totale .....

Esercizio 1 (punti 20)

Si consideri l'astrazione procedurale **raddoppiaArray**. L'astrazione ha come parametro un array x tale che: gli elementi di x sono Integer, x contiene almeno un elemento, e gli elementi di x sono tutti diversi da null. L'astrazione restituisce un array di Integer, di dimensioni raddoppiate rispetto a quelle di x, così costituito:

1) nella metà inferiore dell'array ci sono gli elementi dell'array x (nello stesso ordine);
2) nella metà superiore ci sono solo dei null.

Ad es., se x è [1 2 3 4 5], e si cerca di inserire un nuovo elemento, allora l'invocazione raddoppiaArray(x) restituisce un nuovo array di dimensione 10, così fatto:
[1 2 3 4 5 null null null null null]

a- Si scrivano la pre- e post-condizione del metodo, evidenziando tutti gli elementi significativi

public static Integer[] raddoppiaArray(Integer[] x)

b- Si identifichino le eccezioni necessarie per rendere totale il metodo e si scrivano le opportune modifiche alla specifica di cui al punto (a).

Esercizio 2 (punti 60)

Si consideri la seguente specifica di una classe CodaDoppia.
Una CodaDoppia è una coda in cui gli elementi possono essere inseriti o eliminati in due posizioni, la testa e la coda.

```
public class CodaDoppiaInt{
/*OVERVIEW: Collezione mutabile di oggetti, di tipo Integer, organizzati in una sequenza.
 * Un oggetto tipico di dimensione n è x1 x2 … xn, in cui x1 è la testa e xn è la coda.
*  L'estrazione e l'inserimento di elementi può avvenire sia in testa che in coda.*/

//@ensures (*inizializza this alla CodaDoppiaInt vuota*).
public CodaDoppiaInt ()
//osservatori puri:

//@ensures (* \result è il numero di elementi di this *)
public /*@ pure @*/ int size()

//@ensures (* \result è un ArrayList contenente tutti e soli gli elementi di this *);
public /*@ pure @*/ ArrayList<Integer> elementi()

//@requires size()>0;
//@ensures elementi().get(0).equals(\result);
public /*@ pure @*/ Integer peekFirst()

//@requires size()>0;
//@ensures elementi().lastElement().equals(\result);
public /*@ pure @*/ Integer peekLast()

//mutators:
//@ensures (* aggiunge x in prima posizione *);
public void insertFirst(Integer x)

//@requires size()>0;
//@ ensures (* elimina e restituisce l'elemento in prima posizione in this  *)
public Integer extractFirst()
}
//@ensures (* aggiunge x in ultima posizione *);
public void insertLast(Integer x)

//@requires size()>0;
//@ ensures (* elimina e restituisce l'elemento in ultima posizione in this  *)
public Integer extractLast()
}
```

Domanda A) Specificare in JML le postcondizioni dei metodi insertFirst e extractLast

Si consideri ora la seguente classe CodaInt, che ha gli stessi metodi di CodaDoppiaInt, con la stessa specifica, *ad esclusione* di: peekLast, insertFirst e extractLast, che non vi compaiono.
Per semplicità, non riportiamo la specifica dei metodi di CodaInt, in quanto non modificata rispetto a CodaDoppiaInt.

```
public class CodaInt{
/* OVERVIEW: Collezione mutabile di oggetti, di tipo Integer, organizzati in una sequenza.
 * Un oggetto tipico di dimensione n è x1 x2 … xn, in cui x1 è la testa e xn è la coda. L'estrazione di elementi avviene in testa, mentre l'inserimento avviene in fondo alla coda.*/
    public CodaInt ()
    public /*@ pure @*/ int size()
    public /*@ pure @*/ ArrayList<Integer> elementi()
    public /*@ pure @*/ Integer peekFirst()
    public void insertLast(Integer x)
    public Integer extractFirst()
}
```

Domanda B1): E' possibile definire CodaInt come erede di CodaDoppioInt, rispettando il principio di sostituzione? Giustificare la risposta.

Domanda B2): E' possibile definire CodaDoppioInt come erede di CodaInt, rispettando il principio di sostituzione? Giustificare la risposta.

Si consideri **la classe *CodaInt* della Domanda B.** È data *un'implementazione* realizzata nel modo seguente. Gli elementi sono contenuti in un array *elems*, di dimensione minima 5. Un indice intero *primo* indica la testa e un indice intero *ultimo* indica il fondo della coda memorizzata in elems.
Il rep quindi è:

```
private Integer[] elems;
private int primo;
private int ultimo;
```

CodaInt è implementata inserendo i nuovi elementi nella posizione *ultimo,* ed estraendo gli elementi dalla posizione *primo+1,* aggiornando i valori di primo e di ultimo.
Quando l'array elems non è più sufficiente a contenere gli elementi, si invoca il metodo statico *raddoppiaArray* dell'esercizio 1, che raddoppia la dimensione di elems, copiando inoltre gli elementi esistenti nella metà inferiore dell'array e riempendo il resto con null.

E' data la seguente implementazione del costruttore e dei metodi insertLast e extractFirst:

```
public  CodaInt() {
        elems = new Integer[5];
        primo = -1;
        ultimo=0;
}
public  void insertLast (Integer x) {
        if (ultimo >= elems.length)
                elems = raddoppiaArray(elems);
        elems[ultimo]= x;
        ultimo++;
}
public  Integer extractFirst() {
        if (size()>0) {
                primo++;
                return elems[primo];
        }
}
```

Domanda C) Alla luce della descrizione informale e del codice presentati, scrivere l'invariante di rappresentazione (ossia, privato) di CodaInt.

Domanda D) Descrivere, nel modo che si ritiene più opportuno, la funzione di astrazione di CodaInt.

Domanda E) Scrivere l'implementazione del metodo size(). Argomentare che tale implementazione è corretta rispetto alla sua specifica.

Domanda F) (*Facoltativa*): l'implementazione qui riportata per CodaInt, pur se tecnicamente corretta, soffre di un serio problema, che la rende inadatta a essere utilizzata in pratica. Quale?

Esercizio 3 (punti 20)

Implementare il seguente metodo iteratore *els*() della classe CodaInt dell'Es. 2. Si suggerisce di descrivere la struttura delle eventuali classi aggiuntive rispetto a CodaInt.

//@ensures (restituisce un iteratore agli elementi di this, rispettandone l'ordine di inserimento*);*
public Iterator<Integer> els()